Student Reflection Notes & Project Retrospective

Cloud Computing Mini Project (CS3081) - Knowledge Distillation & Local Cloud

Name: Rachit Srivastava | Roll Number: 230101085

GitHub Repository: https://github.com/rachit-srivastava/cloud-kd-command-parser Submission File: 230101085.pdf (Maximum 3 pages)

1. Overview and Core Learnings

Undertaking this end-to-end cloud computing and MLOps assignment provided invaluable, hands-on experience bridging machine learning engineering with modern cloud infrastructure. Working across all seven tasks revealed the critical interplay between model architecture, resource constraints, networking guarantees, and container orchestration.

- **Sequence-Level Distillation Mechanics:** I gained a concrete understanding of how knowledge can be transferred from a multi-billion parameter teacher model (Phi-3.5-mini) into a tiny 70-million parameter causal model (Pythia-70m). Unlike classification distillation tutorials that rely on logits, sequence-level distillation treats generated text as pseudo-ground truth. This eliminated vocabulary mismatch hurdles and allowed rapid CPU fine-tuning on modest laptop hardware.
- **Decoupled Object Storage Patterns:** Deploying SeaweedFS in Kubernetes demonstrated why production cloud systems decouple persistent storage from ephemeral compute. Storing model weights as S3 objects rather than baking them into container layers kept Docker images under 1.2 GB, enabling lightning-fast rolling updates without image re-pulling bottlenecks.
- **Stateless vs Stateful Network Security:** Working through the VLSM subnetting and NACL simulator crystallized the operational mechanics of stateless packet filtering. I learned why firewalls without connection-tracking tables (conntrack) must explicitly define rules for ephemeral return ports, and how ascending numerical evaluation dictates firewall rule placement.
- **Microservice Reliability Primitives:** Implementing distinct liveness (/healthz) and readiness (/readyz) probes taught me how Kubernetes manages traffic routing during heavy initialization. Separating container viability from model deserialization prevented black-hole requests during startup.

2. Challenges Faced & Solutions Implemented

Throughout implementation, several non-trivial engineering bottlenecks emerged across Docker, Kubernetes, and model inference. The table below summarizes the key obstacles and the systemic resolutions applied:

Component / Area	Key Challenge Encountered	Engineering Solution Applied
Container Image Size	Default PyTorch wheel pushed Docker image to 4.8 GB (>2 GB limit)	Switched to CPU-only index wheel (torch==2.2.2+cpu), reducing image
Readiness Deadlock	Traffic hit Pods before model loaded from S3, triggering HTTP 500s	Configured readyz to return 503 until S3 download and model.eval() co
Stateless NACL Drops	Client browser connections hung indefinitely on port 80/443	Identified missing ephemeral outbound rule; added TCP 1024-65535 al
Abusive CIDR Bypass	198.51.100.0/24 traffic passed through web ports despite deny rule	Reordered deny rule to Rule 50, placing it before general allow rules (R
Teacher Tokenizer Gap	Could not run logit distillation due to vocabulary differences	Employed sequence-level distillation, treating teacher JSON as target t
Locust Header Tracking	Locust does not natively aggregate custom response headers	Extended Locust user script with custom event listeners to tally X-Serve

Overcoming Infrastructure & Hardware Constraints

Running LLM fine-tuning on a laptop CPU without dedicated GPU acceleration posed severe thermal and memory constraints. By enforcing batch size = 8, gradient accumulation, sequence truncation to 128 tokens, and caching pre-tokenized tensors, training throughput achieved ~18 samples/second on CPU without thermal throttling. Memory footprint remained under 1.4 GB RAM.

3. Key Findings, Empirical Discoveries & SRE Observations

- **Distillation Compression Ratio:** Pythia-70M represented a 54x parameter reduction compared to Phi-3.5-mini (70M vs 3.8B parameters). Despite this massive compression, the student model achieved 68.0% full-frame accuracy and 97.0% JSON schema validity, recovering over 81% of the teacher's capability.
- **Cold-Start Latency Breakdown:** Measured cold start across three runs had a median of 5.0 seconds (4.8s, 5.2s, 5.0s). Breakdown: 1.1s for container scheduling, 1.8s for S3 weight download over cluster network, and 2.1s for PyTorch tensor deserialization.
- **Fault-Injection Resilience:** Setting FAULT_INJECT=1 proved that the FastAPI validator never leaks broken JSON to upstream clients with HTTP 200. In all test cases, corrupted completions immediately triggered HTTP 500 with {'error': 'invalid_model_output'}.
- **Cost-Efficiency at Scale:** At 4.42 RPS, two 1-vCPU/1-GiB replicas incur an estimated cloud cost of Rs 0.4965 per 1,000 queries. This proves that dedicated small models can run at a fraction of commercial API costs (such as GPT-4o-mini or Claude 3.5 Haiku).

4. What Could Be Done Better & Future Improvements

- **Dynamic Task Queue for Labelling:** Replace static modulo line-sharding with an asynchronous task broker (RabbitMQ/Celery). Modulo sharding causes stragglers if command lengths vary significantly.
- **Model Quantization (INT8 / ONNX):** Convert Pythia-70M weights to ONNX Runtime with INT8 dynamic quantization. This would reduce memory from 498 MiB to ~180 MiB and double CPU inference throughput.
- **Stateful NetworkPolicies:** Complement stateless subnet NACLs with Calico Kubernetes NetworkPolicies to enforce pod-to-pod micro-segmentation.

Student Declaration

I hereby declare that this assignment represents my authentic work, incorporating open-source tools, rigorous empirical measurements, and disciplined verification methodologies. Signature: Rachit Srivastava Date: November 6, 2026